

Assignment 02: Measuring Runtime with Python

Google Colab, Python, Sorting, and Google Sheets

Name: _____

Date: _____

What Is This Assignment About?

In this assignment, you will use Python to measure how long a program takes to run.

Computers can do many tasks very quickly, but not all tasks take the same amount of time. Some tasks take longer when the amount of data gets larger.

You will run a program that sorts lists of random numbers. You will test different list sizes, record the runtimes, and then make a graph.

Main Question

What happens to the runtime when the list size gets larger?

What You Will Practice

- Opening and using Google Colab
- Running Python code
- Changing a variable in a program
- Measuring runtime using a timer
- Recording data from program output
- Making a graph in Google Sheets
- Submitting a Python `.py` file in Microsoft Teams

What You Need

- A computer, laptop, Chromebook, or iPad with internet access
- Your own personal Gmail email address
- The Google Chrome web browser, updated to the current version
- Access to Microsoft Teams for Education
- Access to Google Sheets

Important: For this class, please use **Google Chrome** to open Google Colab. Do not use Safari unless your teacher gives you permission.

If you are using an iPad, install or update the **Google Chrome** app before starting this assignment.

If you do not have a personal Gmail email address, you need to create one before starting this assignment.

<https://accounts.google.com/signup>

What You Will Submit

You will submit your Python file in Microsoft Teams.

Submit This	Do Not Submit This
RuntimeSorting_YourName.py	Any file ending in .ipynb

Your teacher wants the Python file only. A Python file ends with:

.py

Do **not** submit the Colab notebook file. A notebook file ends with:

.ipynb

Part 1: Open the Starter Notebook

Follow these steps carefully.

1. Open the **Google Chrome** browser.
2. Go to the starter notebook:

https://colab.research.google.com/github/bcc-teachermark/M513-2026/blob/main/Assignment_02.ipynb

3. If Google asks you to sign in, sign in using your own personal Gmail email address.
4. When the notebook opens, click or tap **File**.
5. Choose **Save a copy in Drive**.
6. Work only on your own saved copy.

Check your progress:

- ☐ I opened Google Chrome.
- ☐ I opened the starter Colab notebook.
- ☐ I signed in with my own personal Gmail email address.
- ☐ I saved my own copy in Google Drive.

Part 2: What Is Google Colab?

Google Colab is a website where you can write and run Python code in your web browser.

A Colab notebook is made of boxes called **cells**.

There are two common types of cells:

Cell Type	What It Does
Text cell	Used for notes, explanations, titles, and instructions.
Code cell	Used for writing and running Python code.

For this assignment, you will mostly run and edit **code cells**.

Part 3: Run the Teacher Setup Cell

The first code cell in the notebook is the **Teacher Setup** cell.

This cell loads a teacher-supplied timer called **CodeTimer**. You do not need to understand how **CodeTimer** works inside. You only need to know how to use it.

What CodeTimer Does

CodeTimer measures how long part of a program takes to run.

You will use it like this:

```
timer = CodeTimer("My timing test")

timer.start()

# Code to measure goes here

timer.stop()
```

Run the Setup Cell

1. Find the first code cell.
2. Click or tap the play button on the left side of the cell.
3. Wait until you see a message that says:

CodeTimer is ready to use.

Check your progress:

- ☐ I ran the Teacher Setup cell.
- ☐ I saw the message that **CodeTimer** is ready.

Part 4: First Timing Test

The next code cell is a small first timing test.

It will look similar to this:

```
from codetimer import CodeTimer

timer = CodeTimer("My first timing test")

timer.start()

student_name = "Phat"
print("Hello T. Mark. I am " + student_name + ".")

timer.stop()
```

Your Task

Change this line:

```
student_name = "Phat"
```

so that it uses your own name.

For example:

```
student_name = "Boontam"
```

Then run the cell.

What You Should See

You should see a greeting with your name and a runtime measurement.

Example:

```
Hello T. Mark. I am Boontam.
My first timing test took 123.45 microseconds.
```

Your exact runtime may be different. That is normal.

Check your progress:

- ☐ I changed the student name to my own name.
- ☐ I ran the first timing test.
- ☐ I saw a runtime measurement.

Part 5: The Sorting Runtime Experiment

Now you will measure how long it takes Python to sort lists of random numbers.

A **list** is a group of values. In this assignment, each list contains random integers.

The program will sort the numbers from smallest to largest.

The Experiment Question

As the list size gets larger, what happens to the runtime?

The Code You Will Run

The sorting experiment cell will look similar to this:

```
from codetimer import CodeTimer
import random

def make_random_list(size):
    numbers = []

    for count in range(size):
        random_number = random.randint(1, 1000000)
        numbers.append(random_number)

    return numbers

def selection_sort(numbers):
    for start_index in range(len(numbers)):
        smallest_index = start_index

        for current_index in range(start_index + 1, len(numbers)):
            if numbers[current_index] < numbers[smallest_index]:
                smallest_index = current_index

        numbers[start_index], numbers[smallest_index] = numbers[smallest_index], numbers[start_index]

sizes = [100, 200, 400, 800, 1600]

results = []

print("Sorting Runtime Experiment")
print("-----")
print()

for list_size in sizes:
    numbers = make_random_list(list_size)

    timer = CodeTimer("Sorting " + str(list_size) + " numbers")

    timer.start()

    selection_sort(numbers)

    elapsed_time = timer.stop()

    results.append([list_size, elapsed_time])
```

```

print()

print("Copy this table into Google Sheets:")
print("List Size\tRuntime in Microseconds")

for row in results:
    print(str(row[0]) + "\t" + str(round(row[1], 2)))

```

Important Line

This line controls the list sizes:

```
sizes = [100, 200, 400, 800, 1600]
```

You will run the experiment once using the original sizes. Then you will change this line and run the experiment again.

Part 6: First Run

Run the sorting experiment using the original list sizes:

100, 200, 400, 800, 1600

Record your results below.

List Size	Runtime in Microseconds
100	
200	
400	
800	
1600	

Check your progress:

- ☐ I ran the sorting experiment with the original list sizes.
- ☐ I recorded the runtime results.

Part 7: Second Run

Now change the list sizes.

Find this line in the code:

```
sizes = [100, 200, 400, 800, 1600]
```

Change it to:

```
sizes = [200, 400, 800, 1600, 3200]
```

Run the sorting experiment again.

Record your new results below.

List Size	Runtime in Microseconds
200	
400	
800	
1600	
3200	

Check your progress:

- ☐ I changed the list sizes.
- ☐ I ran the sorting experiment a second time.
- ☐ I recorded the new runtime results.

Part 8: Third Run - Your Choice

Now choose your own list sizes.

Your list should have at least five values.

You may use one of these examples:

```
sizes = [150, 300, 600, 1200, 2400]
```

or

```
sizes = [250, 500, 1000, 2000, 4000]
```

or you may create your own.

My List Sizes

Write the list sizes you chose:

My Third Run Results

List Size	Runtime in Microseconds

Check your progress:

- ☐ I chose my own list sizes.
- ☐ I ran the experiment with my own list sizes.
- ☐ I recorded the runtime results.

Part 9: Make a Graph in Google Sheets

Now you will make a graph of your results.

The graph should show:

List Size vs. Runtime

Steps

1. Open Google Sheets.
2. Create a blank spreadsheet.
3. In cell A1, type:

List Size

4. In cell B1, type:

Runtime in Microseconds

5. Enter the results from one of your runs.
6. Highlight your data.
7. Click or tap **Insert**.
8. Click or tap **Chart**.
9. Choose a **line chart** or **scatter chart**.
10. Make sure the x-axis is **List Size**.
11. Make sure the y-axis is **Runtime in Microseconds**.

What to Look For

Look at the shape of the graph.

Ask yourself:

- Does the runtime increase as the list size gets larger?
- Does the graph look like a straight line?
- Does the graph curve upward?
- Did doubling the list size exactly double the runtime?

Check your progress:

- ☐ I entered my data into Google Sheets.
- ☐ I made a graph.
- ☐ My graph shows list size and runtime.

Part 10: Reflection Questions

Answer these questions in complete sentences.

1. What happened to the runtime as the list size became larger?
2. Did doubling the list size exactly double the runtime? Explain your answer.
3. Which list size took the longest to sort?
4. Why might your runtime results be different from another student's results?
5. Look at your graph. Does the runtime grow in a straight line, or does it curve upward?
6. What is one thing you changed in the Python code yourself?

Part 11: iPad Tips

If you are using an iPad, these tips will help.

- Use the **Chrome** app, not Safari.
- Turn your iPad sideways into **landscape mode**. This gives you more space.
- Tap inside a code cell before typing.
- Tap the play button to run the code.
- If the keyboard covers part of the screen, scroll slightly or turn the iPad sideways.
- If you have an external keyboard for your iPad, you may use it, but it is not required.
- Make sure your iPad is connected to Wi-Fi.

Part 12: Common Problems and Fixes

Problem	What to Check
I am using Safari.	Close Safari and open Google Colab in the Chrome app instead.
I cannot edit the notebook.	Make sure you chose File then Save a copy in Drive . Work on your own copy.
Nothing happens when I type.	Make sure you clicked or tapped inside a code cell.
The play button is missing.	Click or tap inside the code cell. The play button usually appears on the left.
I see an error message.	Check spelling, quotation marks, parentheses, brackets, and commas. Small details matter in programming.
CodeTimer is not found.	Go back and run the Teacher Setup cell again.
My graph looks different from another student's graph.	That is normal. Different computers and Colab sessions may produce different runtimes. Look for the general pattern.
I downloaded the wrong file.	You must download the <code>.py</code> file, not the <code>.ipynb</code> notebook file.

Part 13: Download Your Python File

When your work is finished, download your Python file from Google Colab.

1. In Google Colab, click or tap **File**.
2. Click or tap **Download**.
3. Choose **Download .py**.
4. Wait for the file to download.

Your downloaded file should end with:

`.py`

Rename the file using this format:

`RuntimeSorting_YourName.py`

Example:

`RuntimeSorting_Phathat.py`

Important: Do not submit a file ending in `.ipynb`.

Check your progress:

- ☐ I clicked or tapped **File**.
- ☐ I clicked or tapped **Download**.
- ☐ I selected **Download .py**.
- ☐ I checked that my file ends with `.py`.

Part 14: Submit in Microsoft Teams

Submit your assignment in Microsoft Teams for Education.

1. Open **Microsoft Teams**.
2. Go to the class team for this course.
3. Open this assignment.
4. Choose the option to **attach** or **upload** your work.
5. Attach your downloaded Python file.
6. Check that the attached file ends with `.py`.
7. Submit or turn in the assignment.

Do not attach the Colab notebook file.

Correct	Incorrect
<code>RuntimeSorting_YourName.py</code>	<code>Assignment_02.ipynb</code>

Final Checklist

Before you submit, make sure you completed each item.

- ☐ I opened the starter notebook in Google Colab.
- ☐ I saved a copy in Google Drive.
- ☐ I ran the Teacher Setup cell.
- ☐ I changed the first timing test to use my own name.
- ☐ I ran the sorting experiment with the original list sizes.
- ☐ I changed the list sizes and ran the experiment again.
- ☐ I chose my own list sizes and ran the experiment a third time.
- ☐ I recorded my runtime results.
- ☐ I made a graph in Google Sheets.
- ☐ I answered the reflection questions.
- ☐ I downloaded my work as a `.py` file.
- ☐ I submitted only the `.py` file in Microsoft Teams.

Teacher Check

Requirement	Points
Correct .py file submitted, not .ipynb	2
First timing test runs and uses student's own name	1
Sorting experiment runs successfully	2
Student tested at least three sets of list sizes	2
Runtime results were recorded	1
Google Sheets graph was created	1
Reflection questions were completed	1
Total	10